

Experiment 13

Construct a C program for implementation of the various memory allocation strategies.

C Program

```
#include <stdio.h>

void firstFit(int block[], int m, int process[], int n) {
    int alloc[n];
    for (int i = 0; i < n; i++) alloc[i] = -1;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (block[j] >= process[i]) {
                alloc[i] = j;
                block[j] -= process[i];
                break;
            }

    printf("\nFirst Fit:\n");
    for (int i = 0; i < n; i++)
        printf("Process %d -> %s\n", i + 1,
            alloc[i] == -1 ? "Not Allocated" : "Block allocated");
}

void bestFit(int block[], int m, int process[], int n) {
    int alloc[n];
    for (int i = 0; i < n; i++) alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int best = -1;
        for (int j = 0; j < m; j++)
            if (block[j] >= process[i] &&
                (best == -1 || block[j] < block[best]))
                best = j;
        if (best != -1) {
            alloc[i] = best;
            block[best] -= process[i];
        }
    }

    printf("\nBest Fit:\n");
    for (int i = 0; i < n; i++)
        printf("Process %d -> %s\n", i + 1,
            alloc[i] == -1 ? "Not Allocated" : "Block allocated");
}

void worstFit(int block[], int m, int process[], int n) {
    int alloc[n];
    for (int i = 0; i < n; i++) alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++)
            if (block[j] >= process[i] &&
                (worst == -1 || block[j] > block[worst]))
                worst = j;
        if (worst != -1) {
            alloc[i] = worst;
            block[worst] -= process[i];
        }
    }

    printf("\nWorst Fit:\n");
    for (int i = 0; i < n; i++)
        printf("Process %d -> %s\n", i + 1,
            alloc[i] == -1 ? "Not Allocated" : "Block allocated");
}

int main() {
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int block1[m], block2[m], block3[m], process[n];
```

```

printf("Enter block sizes:\n");
for (int i = 0; i < m; i++) {
    scanf("%d", &block1[i]);
    block2[i] = block3[i] = block1[i];
}

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter process sizes:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &process[i]);

firstFit(block1, m, process, n);
bestFit(block2, m, process, n);
worstFit(block3, m, process, n);

return 0;
}

```

Sample Output

```

Enter number of memory blocks: 3
Enter block sizes:
100 500 200
Enter number of processes: 3
Enter process sizes:
212 417 112

```

```

First Fit:
Process 1 -> Block allocated
Process 2 -> Block allocated
Process 3 -> Block allocated

```

```

Best Fit:
Process 1 -> Block allocated
Process 2 -> Block allocated
Process 3 -> Block allocated

```

```

Worst Fit:
Process 1 -> Block allocated
Process 2 -> Block allocated
Process 3 -> Block allocated

```

Result

The First Fit, Best Fit and Worst Fit memory allocation strategies were successfully implemented and simulated.